

Name: Nishi Chopda
Jeff Bezos
150096725128

INVENTORY MANAGEMENT SYSTEM

1. Project Overview

The Inventory Management System is a REST-based backend application developed for a retail store to manage products, categories, stock quantities and inventory activity. The backend is built using Node.js and Express.js and uses MongoDB Atlas as the database through Mongoose.

Problem Statement

A retail store needs a system to track product stock across categories. Staff members should be able to view and update stock quantities, while category management is restricted to administrators. The system should also identify products whose stock is at or below their reorder level.

Objectives

- Design Product and Category schemas using Mongoose.
- Implement CRUD operations for products and categories.
- Implement JWT-based authentication and role-based authorization.
- Allow staff users to update product stock.
- Provide a threshold-based low-stock query.
- Prevent invalid negative stock quantities.
- Record stock movements and activity logs.
- Connect the backend to MongoDB Atlas and deploy the API on Render.

2. Technology Stack

Technology	Purpose
Node.js	JavaScript runtime used to execute the backend.
Express.js	Web framework used to create REST API routes and middleware.
MongoDB Atlas	Cloud-hosted NoSQL database.
Mongoose	ODM used for schemas, validation, relationships and database operations.
JWT	Authentication mechanism for protected API requests.
bcryptjs	Password hashing and password verification.
CORS	Allows the deployed frontend to communicate with the backend.
dotenv	Loads environment variables such as MongoDB URI and JWT secret.
Postman	API testing and request collection.

3. Backend Architecture

The backend follows a modular Express.js structure:

Client / React Frontend

|

| HTTP / REST API

v

Express.js Server

|

+-- Routes

|

+-- Authentication & Authorization Middleware

|

+-- Controllers

|

+-- Mongoose Models

|

MongoDB Atlas

Routes define the available API endpoints. Middleware handles authentication, authorization and common error processing. Controllers contain the application/business logic, while Mongoose models define the MongoDB document structure and validation rules.

4. Backend Folder Structure

```
backend/
├── config/
│   └── db.js
├── controllers/
│   ├── authController.js
│   ├── categoryController.js
│   ├── productController.js
│   ├── stockMovementController.js
│   └── activityLogController.js
├── middleware/
│   ├── authMiddleware.js
│   ├── roleMiddleware.js
│   └── errorMiddleware.js
├── models/
│   ├── User.js
│   ├── Category.js
│   ├── Product.js
│   ├── StockMovement.js
│   └── ActivityLog.js
├── routes/
│   ├── authRoutes.js
│   ├── categoryRoutes.js
│   ├── productRoutes.js
│   ├── stockMovementRoutes.js
│   └── activityLogRoutes.js
└── server.js
```

- seed.js
- package.json
- .env.example

5. Database Design

5.1 User Schema

Field	Type	Validation / Purpose
name	String	Required
email	String	Required and unique
password	String	Stored as a bcrypt hash
role	String	Admin or Staff; default Staff
timestamps	Date	Created/updated timestamps

New registrations are always assigned the Staff role. Admin privileges are not accepted from the public registration request.

5.2 Category Schema

Field	Type	Validation / Purpose
name	String	Required and unique
description	String	Optional
timestamps	Date	Created/updated timestamps

5.3 Product Schema

Field	Type	Validation / Purpose
name	String	Required, trimmed
sku	String	Required, unique, trimmed
description	String	Optional
price	Number	Required, minimum 0
stockQuantity	Number	Required, minimum 0
reorderLevel	Number	Required, minimum 0; default 10
category	ObjectId	Required reference to Category

timestamps	Date	Created/updated timestamps
------------	------	----------------------------

5.4 StockMovement Schema

Field	Type	Purpose
product	ObjectId	Referenced product
previousQuantity	Number	Stock before update
newQuantity	Number	Stock after update
quantityChanged	Number	Difference between new and previous stock
reason	String	Restock, Sale, Damaged, Returned or Manual Adjustment
performedBy	ObjectId	User who performed the update
timestamps	Date	Movement time

5.5 ActivityLog Schema

Field	Type	Purpose
user	ObjectId	User who performed the action
action	String	Description of action
entity	String	Product, Category or Stock
entityId	ObjectId	Related entity
details	String	Additional information
timestamps	Date	Activity time

6. Authentication and Authorization

Registration

POST /api/auth/register

```
{
  "name": "Staff User",
  "email": "staff@example.com",
  "password": "password"
}
```

The password is hashed using bcryptjs before storage. The role is always set to Staff.

Login

POST /api/auth/login

After successful password verification, the backend creates a JWT containing the user ID and role. The token expires after one day.

Authorization: Bearer <JWT_TOKEN>

Role-based Authorization

The protect middleware verifies the JWT. The allowRoles middleware checks the role contained in the authenticated request. Category creation, modification and deletion are restricted to Admin users.

7. REST API Endpoints

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Register a Staff user
POST	/api/auth/login	Public	Authenticate and receive JWT
GET	/api/categories	Authenticated	List categories
POST	/api/categories	Admin	Create category
PATCH	/api/categories/:id	Admin	Update category
DELETE	/api/categories/:id	Admin	Delete category
GET	/api/products	Authenticated	List/search/filter products
GET	/api/products/stats	Authenticated	Get inventory statistics
GET	/api/products/reorder	Authenticated	Get reorder recommendations
GET	/api/products/:id	Authenticated	Get one product
POST	/api/products	Authenticated	Create product
PATCH	/api/products/:id	Authenticated	Update product

PATCH	/api/products/:id/stock	Authenticated	Update stock quantity
DELETE	/api/products/:id	Authenticated	Delete product
GET	/api/stock-movements	Authenticated	View recent stock movements
GET	/api/activity-logs	Authenticated	View recent activity logs

8. Important Backend Features

8.1 Low-stock Query

The backend supports a threshold-based low-stock filter using the condition `stockQuantity <= reorderLevel`.

GET /api/products?lowStock=true

The same condition is used by the statistics and reorder recommendation endpoints.

8.2 Stock Validation

Negative stock quantities are rejected when creating products, updating products and updating stock.

8.3 Stock Movement Tracking

When stock is updated, the system stores the previous quantity, new quantity, quantity difference, reason and user who performed the operation.

8.4 Reorder Recommendations

Products at or below their reorder level are returned with the current stock, reorder level and recommended quantity.

9. Middleware

Middleware	Function
authMiddleware.js	Reads Bearer JWT, verifies it and attaches authenticated user information to req.user.

roleMiddleware.js	Restricts selected routes to specified roles such as Admin.
errorMiddleware.js	Handles unhandled server errors and returns a consistent JSON error response.

10. Validation and Error Handling

- Required fields are checked before creating or updating records.
- Duplicate category names and product SKUs are rejected.
- Invalid category references are rejected.
- Negative price, stock and reorder values are prevented by controller checks and Mongoose validation.
- Missing or invalid JWTs return HTTP 401.
- Insufficient role permissions return HTTP 403.
- Missing resources return HTTP 404.
- Unexpected server/database errors return HTTP 500.

11. Environment Configuration

The backend uses environment variables instead of hardcoding deployment credentials.

```
MONGO_URI=your_mongodb_connection_string
PORT=3000
JWT_SECRET=your_jwt_secret
```

The real .env file is not part of the public source configuration. The repository contains .env.example as a safe template.

12. Deployment

MongoDB Atlas

MongoDB Atlas hosts the production database. The backend connects using the MONGO_URI environment variable. The Atlas network access configuration allows the deployed Render service to reach the cluster.

Render

The Express backend is deployed as a Render Web Service from the backend directory.

Setting	Value
Root Directory	backend
Build Command	npm install
Start Command	node server.js
Database	MongoDB Atlas
Environment Variables	MONGO_URI, JWT_SECRET

Live backend URL:

<https://inventorymanagementsystem-4vkt.onrender.com>

13. API Testing

The project includes a Postman collection for testing authentication, categories, products, stock updates, stock movements and activity logs.

Test Area	Expected Result
Admin login	JWT token returned
Staff login	JWT token returned
Invalid login	401 response
Admin creates category	201 response
Staff creates category	403 response
Product creation	201 response
Negative stock update	400 response
Low-stock query	Products at or below reorder level returned
Invalid product/category ID	Appropriate error response
Stock update	Product and stock movement record updated

14. Example Stock Update Flow

1. Client sends PATCH /api/products/:id/stock
2. protect middleware verifies JWT
3. Controller validates stockQuantity and reason
4. Product is loaded from MongoDB

5. Previous stock quantity is stored
6. Product stockQuantity is updated
7. StockMovement document is created
8. ActivityLog document is created
9. Updated product is returned to the client

15. Security Considerations

- Passwords are hashed using bcryptjs and are never returned in login/register responses.
- Protected endpoints require a valid Bearer JWT.
- Admin-only category operations use role-based middleware.
- Public registration cannot assign the Admin role.
- Database credentials and JWT secrets are supplied through environment variables.
- The frontend communicates with the backend through the deployed API URL rather than embedding database credentials.

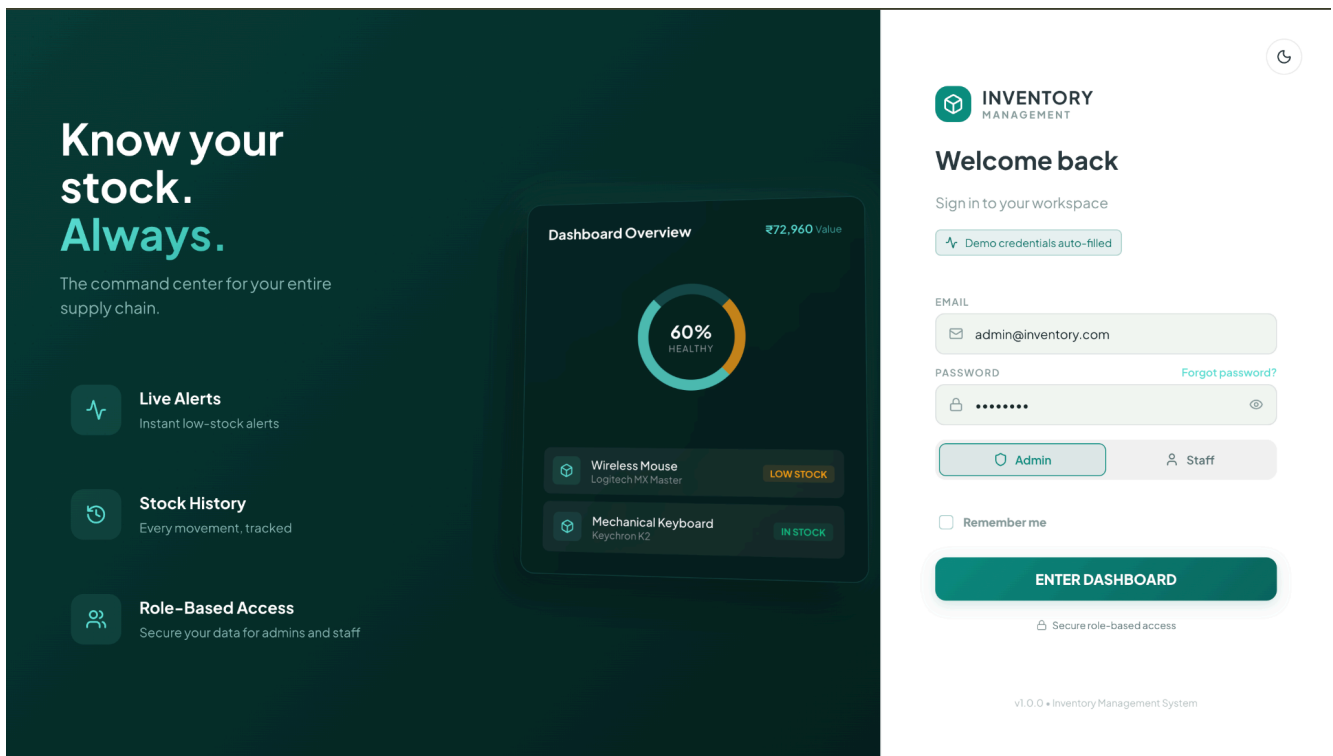
18. Conclusion (Reference)

The Inventory Management System backend provides a complete REST API for inventory management using Node.js, Express.js, MongoDB Atlas and Mongoose. It implements authentication, role-based authorization, CRUD operations, stock validation, low-stock detection, stock movement tracking and activity logging. The backend is deployed on Render and is designed to communicate with the React frontend deployed separately.

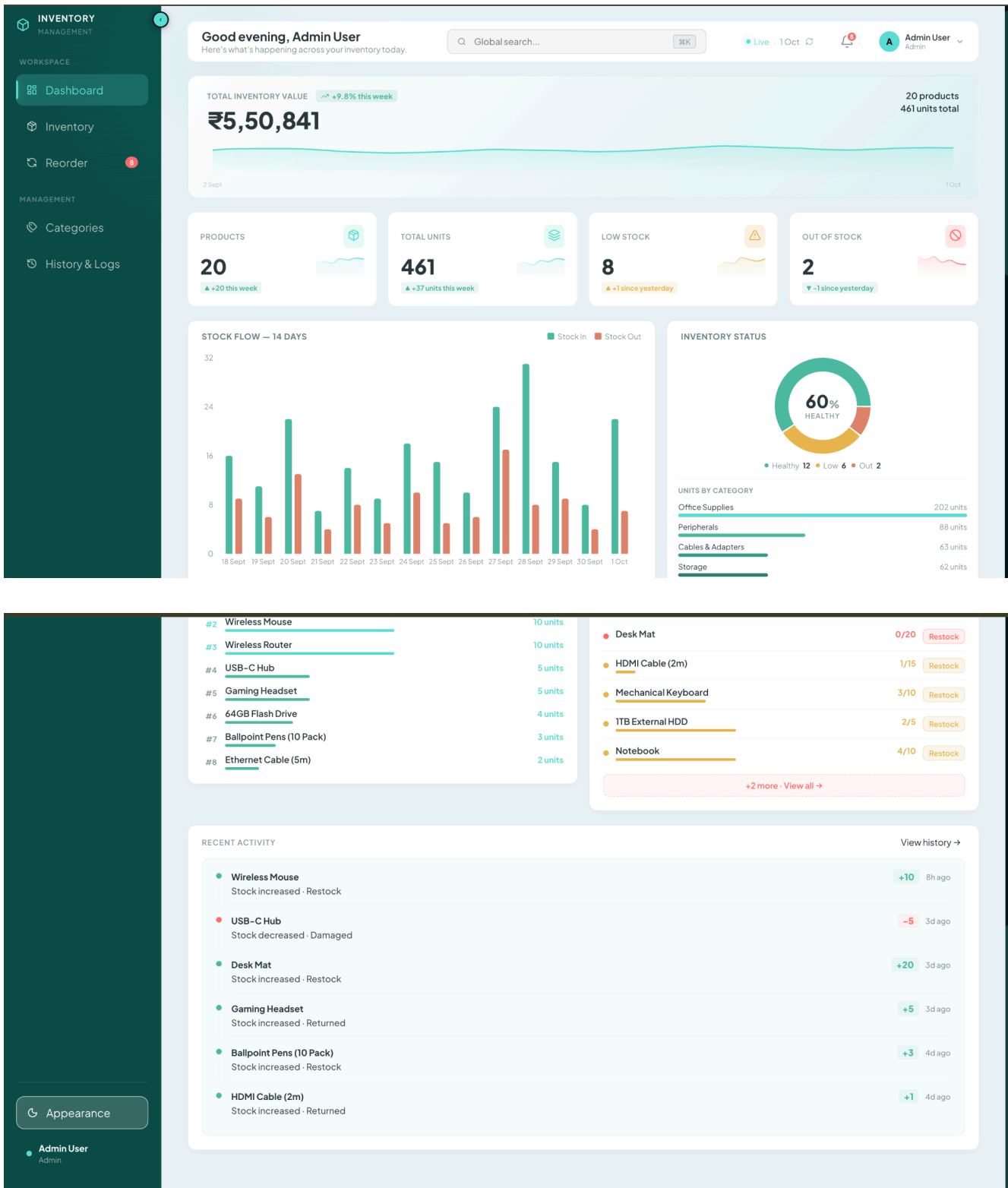
16. UI Screenshots

The following section is reserved for screenshots of the deployed Inventory Management System interface. Screenshots can be inserted below each heading to demonstrate the frontend screens connected to the backend API.

16.1 Login Page



16.2 Dashboard



16.3 Inventory / Products

INVENTORY MANAGEMENT

WORKSPACE

Dashboard

Inventory

Reorder

MANAGEMENT

Categories

History & Logs

Inventory

Manage and track your product inventory.

Global search...

Live · 2 Oct

Admin User

20 PRODUCTS

Export CSV

Add Product

Search products or SKU...

STATUS: ALL 20 IN STOCK 12 LOW STOCK 6 OUT OF STOCK 2

CATEGORY: All Electronics Peripherals Cables & Adapters Storage Office Supplies

	PRODUCT	SKU	CATEGORY	STATUS	PRICE	STOCK	
☐	1TB External HDD	ST0-001	Storage	Low Stock	₹4,599	2 / min 5	<button>Update Stock</button>
☐	24-inch Monitor	ELE-002	Electronics	In Stock	₹12,500	12 / min 5	<button>Update Stock</button>
☐	256GB SSD	ST0-002	Storage	In Stock	₹2,899	25 / min 10	<button>Update Stock</button>
☐	64GB Flash Drive	ST0-003	Storage	In Stock	₹699	35 / min 15	<button>Update Stock</button>
☐	Ballpoint Pens (10 Pack)	OFF-003	Office Supplies	In Stock	₹150	150 / min 50	<button>Update Stock</button>

Appearance

Admin User

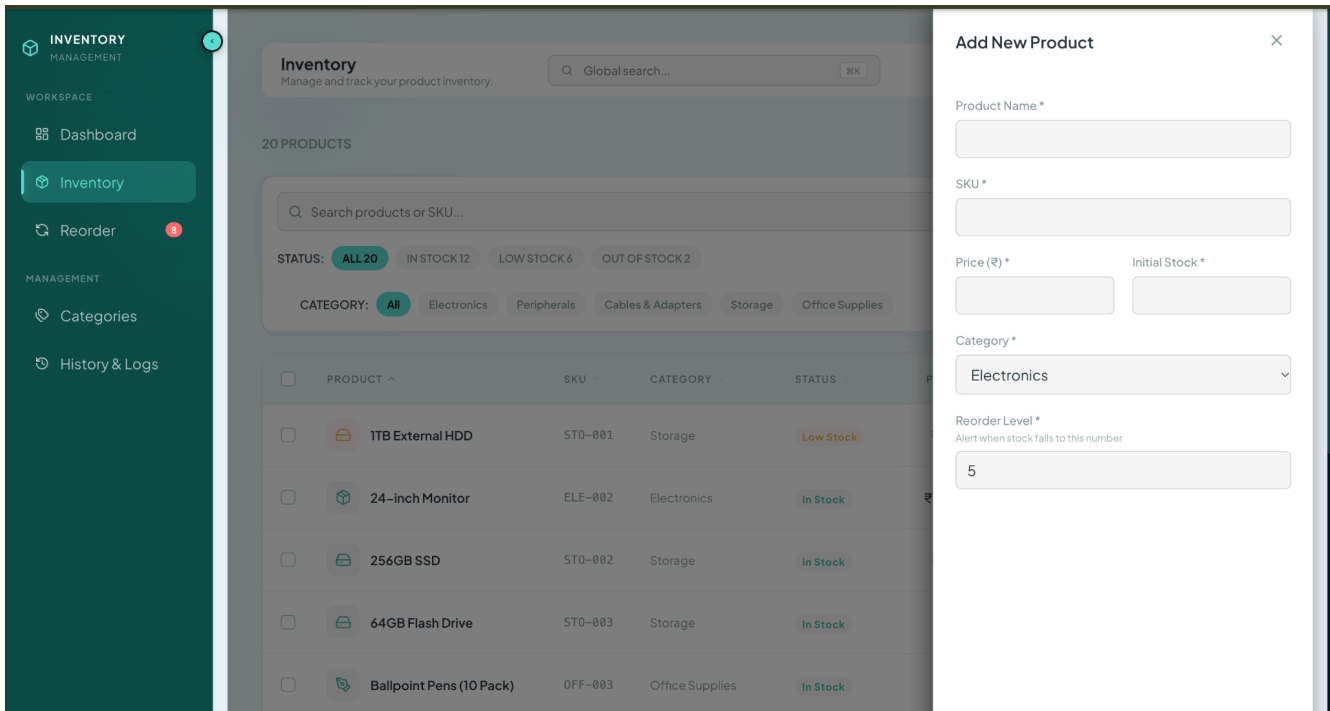
Admin

☐	1TB External HDD	ST0-001	Storage	Low Stock	₹4,599	2 / min 5	<button>Update Stock</button>
☐	24-inch Monitor	ELE-002	Electronics	In Stock	₹12,500	12 / min 5	<button>Update Stock</button>
☐	256GB SSD	ST0-002	Storage	In Stock	₹2,899	25 / min 10	<button>Update Stock</button>
☐	64GB Flash Drive	ST0-003	Storage	In Stock	₹699	35 / min 15	<button>Update Stock</button>
☐	Ballpoint Pens (10 Pack)	OFF-003	Office Supplies	In Stock	₹150	150 / min 50	<button>Update Stock</button>
☐	Bluetooth Speaker	ELE-001	Electronics	Low Stock	₹1,999	2 / min 5	<button>Update Stock</button>
☐	Desk Mat	OFF-001	Office Supplies	Out of Stock	₹899	0 / min 20	<button>Update Stock</button>
☐	DisplayPort Cable	CAB-004	Cables & Adapters	In Stock	₹499	22 / min 10	<button>Update Stock</button>
☐	Ergonomic Chair	OFF-005	Office Supplies	In Stock	₹7,999	8 / min 5	<button>Update Stock</button>
☐	Ethernet Cable (5m)	CAB-003	Cables & Adapters	In Stock	₹399	40 / min 20	<button>Update Stock</button>

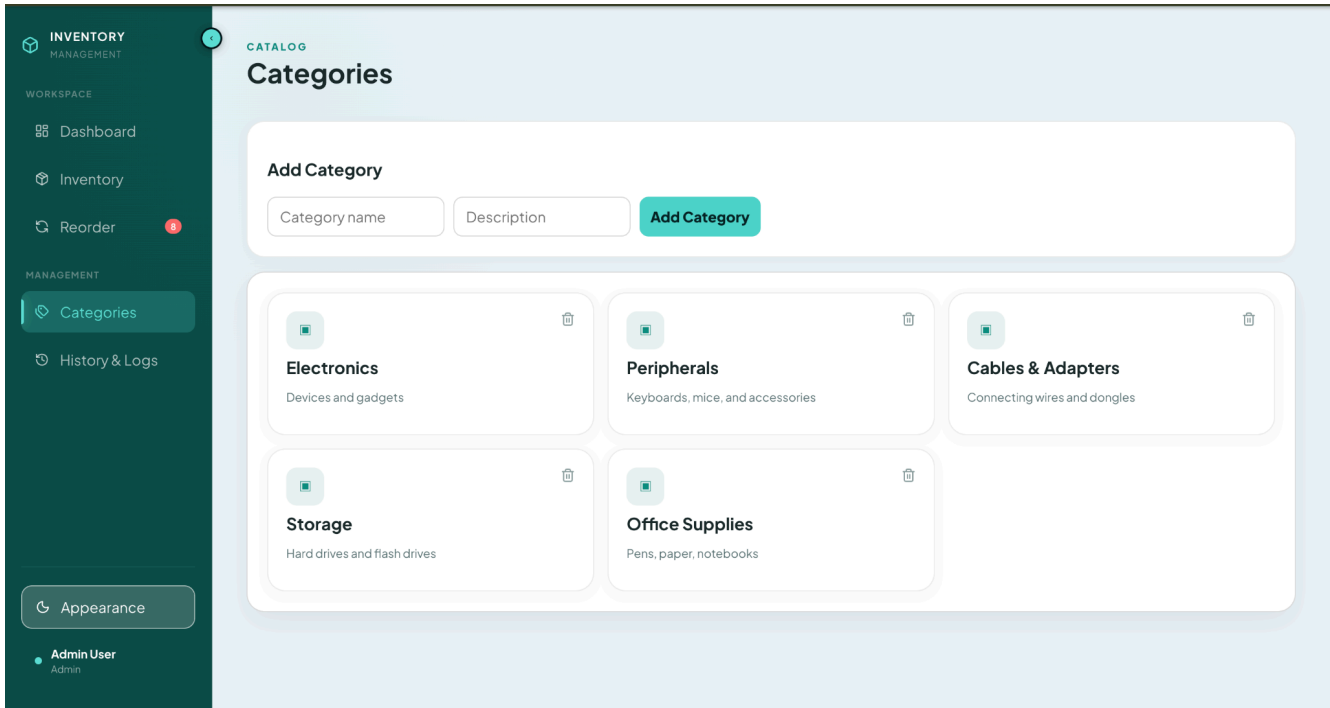
Showing 1 to 10 of 20 Rows per page: 10

1 2

16.4 Add / Edit Product



16.5 Categories



16.6 Reorder Center / Low Stock

INVENTORY MANAGEMENT

WORKSPACE

Dashboard

Inventory

Reorder

MANAGEMENT

Categories

History & Logs

Reorder Center

Manage items that are low or out of stock.

Search products...

All Needs (8)Out of Stock (2)Low Stock (6)

OUT OF STOCK

USB-C Hub

CAB-001

Current: 0Reorder at: 5

SUGGESTED +10 unitsRestock

OUT OF STOCK

Desk Mat

OFF-001

Current: 0Reorder at: 20

SUGGESTED +40 unitsRestock

LOW STOCK

HDMI Cable (2m)

CAB-002

Current: 1Reorder at: 15

SUGGESTED +29 unitsRestock

LOW STOCK

1TB External HDD

STO-001

Current: 2Reorder at: 5

SUGGESTED +8 unitsRestock

LOW STOCK

Bluetooth Speaker

ELE-001

Current: 2Reorder at: 5

SUGGESTED +8 unitsRestock

LOW STOCK

Mechanical Keyboard

PER-002

Current: 3Reorder at: 10

SUGGESTED +17 unitsRestock

16.7 Stock History

INVENTORY MANAGEMENT

WORKSPACE

Dashboard

Inventory

Reorder

MANAGEMENT

Categories

History & Logs

History & Logs

Track stock movements and system activity

Export CSV

Stock MovementsSystem Activity

Search products...

All TimeAll UsersAll ReasonsTableTimeline

PRODUCT	CHANGE	REASON	PERFORMED BY	TIME
Yesterday				
Wireless Mouse PER-001	+10	Restock	Staff User	13:30
28 Sept 2026				
USB-C Hub CAB-001	-5	Damaged	Admin User	21:36
Desk Mat OFF-001	+20	Restock	Admin User	19:26
Gaming Headset PER-003	+5	Returned	Admin User	16:24
27 Sept 2026				
Ballpoint Pens (10 Pack) OFF-003	+3	Restock	Admin User	18:26

16.8 Activity Log

History & Logs			
Track stock movements and system activity			
Stock Movements System Activity			
Search actions... All Time All Users All Reasons Table Timeline			
ACTION	DETAILS	PERFORMED BY	TIME
Yesterday			
+ Updated stock	Stock for Wireless Mouse changed by +10 (Restock)	Staff User	13:30
28 Sept 2026			
⚠ Stock Updated	Updated stock for USB-C Hub by -5 (Damaged)	Admin User	21:36
+ Stock Updated	Updated stock for Desk Mat by +20 (Restock)	Admin User	19:26
🔄 Stock Updated	Updated stock for Gaming Headset by +5 (Returned)	Admin User	16:24
27 Sept 2026			
+ Stock Updated	Updated stock for Ballpoint Pens (10 Pack) by +3 (Restock)	Admin User	18:26
🔄 Stock Updated	Updated stock for HDMI Cable (2m) by +1 (Returned)	Admin User	14:14

17. Conclusion

The Inventory Management System backend provides a complete REST API for inventory management using Node.js, Express.js, MongoDB Atlas and Mongoose. It implements authentication, role-based authorization, CRUD operations, stock validation, low-stock detection, stock movement tracking and activity logging. The backend is deployed on Render and is designed to communicate with the React frontend deployed separately.